

YAZILIM MÜHENDİSLİĞİ

HAZIRLAYAN VE SUNAN

Doç. Dr. Asım Sinan YÜKSEL



asimyuksel@sdu.edu.tr

5. Ders



SÜLEYMAN
DEMİREL
ÜNİVERSİTESİ



BİLGİSAYAR
MÜHENDİSLİĞİ

Önbellek Kavramı

- Önbellekleme uygulamaların cevap verme sürelerini hızlandırmak, sık ihtiyaç duyulan verileri uygulama tarafına yaklaştırarak uygulamaların ölçeklenebilmesini sağlamak için kullanılan bir tekniktir.
- Veri tekrarını önler ve dış kaynaklara erişimi azaltır.
- Uygulamaların o an bulunduğu durumdan bağımsız çalışabilir.
- Web olsun, masaüstü ya da mobil olsun uygulamalar verilerden bağımsız düşünülemez. Girdiler, çıktılar, durumlar bir yerlerde veri olarak saklanmaktadır.
- Veri uygulamanın ya da iş parçalarının ne kadar yakınında durursa, uygulamanın hızı da o kadar yüksek olacaktır.
- Modern bilgisayar sistemlerinde iş parçacıkları/işlemler donanımsal düzeyde ya da küme (cluster) gibi üst düzey soyut alanlarda yani çok farklı katmanlarda çalışmaktadır.

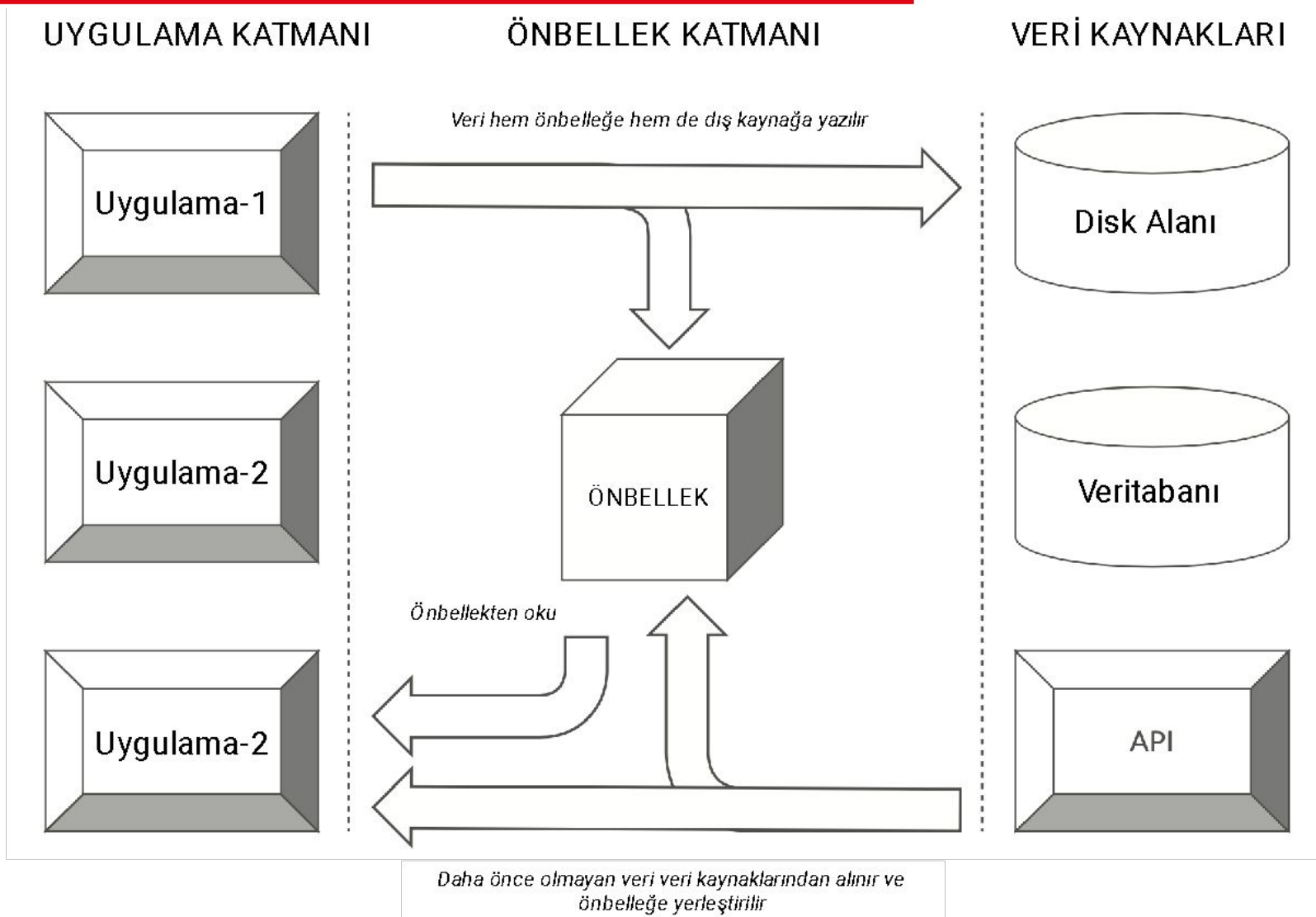
Önbellek Kavramı

- Tipik bir uygulama sunucusunda çalışan işlemleri düşünersek, bu tarz bir uygulamayı 3 aşamaya bölebiliriz:
 - 1. İstemciden gelen istekleri alma
 - 2. Cevabı hazırlamak için gelen isteği işleme
 - 3. Cevabı döndürme
- 2. aşamadaki herhangi bir zamanda veri gerekebilir.
- İşlemin cevabın son halini üretmesi için kendisi ek veri üretebilir.
- Sunucunun RAM'i bu verileri depolamak için kullanılır.
- Ancak tüm veriler RAM'in sınırlarından dolayı burada depolanamaz.
- Bazen veritabanı gibi dış kaynaklarda depolanması gerekebilir.
- Uygulama veritabanı gibi dış kaynakları kullanmaya başladığında sıkıntılar da ortaya çıkmaya başlar.

Önbellek Kavramı

- Yavaş bir dış kaynak kullanıldığında, uygulama performansını artırmak için sık erişilen veri geçici olarak uygulamaya yakın hızlı bir depolama alanına aktarılır.
- Bu aradaki hızlı depolama alanına uygulamanın önbelleği denir.
- Önbelleğin temel amacı uygulamanın kullandığı ana hafıza dışında depolanan veriye daha kısa sürede erişmektir.
- Önbellek olmadan uygulama her aşamada dış kaynaklarla iletişime geçip sürekli veri çekmek zorunda kalacaktır.
- Önbellek kullanıldığında ise tek bir istek yapmak yeterli olacak, daha sonraki erişimler önbelleğe gidilerek yapılacaktır.
- Önbellek aynı zamanda uygulamanın ayakta kalma süresini ve erişilebilirliğini de artıracaktır. Dış kaynaklara erişimde sıkıntı olması durumunda uygulama önbelleği kullanarak çalışmasına devam edebilecektir.

Önbellek Kavramı



Önbellek Kavramı

- Uygulamalar önbelleği çeşitli amaçlar için kullanabilir:
 - **Konfigürasyon ayarları:** Uygulama çalışmaya başlarken ya da çalışırken gerekli bilgilerin depolanması. Bu bilgiler genelde yavaş olan diskte text dosyası şeklinde tutulur. Dolayısıyla önbellekten getirmek başlangıç hızını artıracaktır.
 - **Uygulama Yerelleştirmesi:** Uygulama arayüzlerinin farklı dillere desteklemesi için genelde diskte farklı dil dosyaları oluşturulur. Bu bilginin önbellekte tutulması ve buradan getirilmesi cevap süresini kısaltacaktır.
 - **Şablonlar:** Çoğu uygulama ürettikleri cevabı hazır şablonlara (HTML fragmentleri ya da JSON snippetleri gibi) aktarırlar. Veri dinamiktir ancak bu şablonlar statiktir. Statik verileri önbellekleme bunlara erişim maliyetini azaltacaktır.

Önbellek Kavramı

- Uygulamalar önbelleği çeşitli amaçlar için kullanılabilir (Devam):
 - **Aşırı işlem gücü gerektiren fonksiyonların yeniden kullanılabilir sonuçları:** Bazen uygulamaların aşırı kaynak kullanımı yapması ve sürekli sonuç üretmesi gerekebilir. Bazı durumlarda üretilen sonuçların yeniden kullanılması gerekebilir. Bu tarz ara sonuçların kullanımını hızlandırmak için önbellek kullanılabilir.
 - **Oturum (Session) verileri:** Oturum verilerinin önbelleklenmesi hızlı cevap veren uygulamaların geliştirilmesinde olmazsa olmaz özelliklerden biridir. Kullanıcının her bir etkileşimi oturum verisi gerektirdiğinden bu bilginin ön bellekte tutulması, uygulamanın cevap verme süresini kısaltacaktır.

Önbellek Kavramı

- Uygulamalar önbelleği çeşitli amaçlar için kullanılabilir (Devam):
 - **Veritabanı verisi:** Önbellekleme veritabanı düzeyinde arama tablolarının(lookup table) kopyalarını tutmak için kullanılabilir. Bu tablolar genelde uygulamanın performansını artırmak amacıyla pahalı sorgular için kullanılır.
 - **API cevapları:** Modern uygulamalar API'lerle haberleşecek şekilde tasarlanmıştır. Bu haberleşmede kısa bir süre için bile olsa cevapların önbellekte depolanması iç haberleşme performansını artırabilir.
 - **Uygulama nesneleri:** Daha sonrasında tekrar kullanılacak uygulamalar tarafından oluşturulan nesneler önbellekte tutulabilir.

Önbellek Kavramı

- Önbellekler özel, paylaşımlı, dağıtık paylaşımlı olmak üzere üç gruba ayrılır.
- **Özel Önbellek:**
 - Bir işleme ait olan bir bellek sadece o işlem tarafından kullanılabilir.
 - Bu tarz belleğin temel dezavantajı işlemle aynı RAM'i paylaşıyor olmasıdır.
 - Çünkü bu tarz bir durum çekişmeye neden olacaktır ve genelde işlem kazanan olur.
 - Özel bellekler aynı zamanda ölçeklenebilir değildir.
 - İşleme özel olmaları işlemlerin kopyalarının oluşturulduğu durumlarda verinin de kopyasının oluşturulması problemine neden olacaktırlar.

Önbellek Kavramı

- **Paylaşımlı Önbellek:**

- Veri işlemin kullandığı bellek dışında tutulur.
- Bu durum erişim maliyetini biraz artırsa da uygulama ile önbelleğin kaynak gereksinimleri arasındaki bağı zayıflatacaktır.
- Erişim maliyeti kaçınılmazdır çünkü işlem önbelleğe erişmekten ziyade daha çok isteğe cevap verme ile ilgilenecektir.
- Paylaşımlı bellekler genelde aynı sunucu üzerinde yer almaktadır.
- Sunucuda uygulamanın farklı kopyaları çalışıyorsa aynı önbellek farklı kopyalar tarafından paylaşılabilir.

- **Paylaşımlı Dağıtık Önbellek:**

- Bu tarz bellekler genelde uzak bir sunucu da ya da sunucu kümelerinde yer alır.
- Bu da önbelleğe alınmış verinin gereksiz kopyasının oluşturulmasını önler. İsteklerin durum bilgilerinin de önbellekte tutulması uygulamaların ölçeklenebilirliğini artırır.

Redis

- Redis açık kaynak kodlu bellek içi veri yapısı deposudur.
- Veritabanı, önbellek katmanı ya da mesaj aracısı (message broker) olarak kullanılabilir.
- Redis veri yapıları basit komutlarla karmaşık programlama problemlerini çözebilecek güçtedir. Bu sayede programlama için harcanan çabayı azaltır, çıktıyı artırır ve gecikmeyi azaltır.
- Önbellekleme Redis'in en popüler kullanım alanıdır.
- Redis'in uygulamalarda paylaşımlı dağıtık önbellekleme katmanı olarak kullanılması için gerekli tüm gereksinimleri sağlayacak kapasitededir.
- Bu gereksinimleri 15 başlık altında toplayabiliriz:
 - 1. Bellek içi depolama: Redis verileri RAM'da tutar.
 - 2. Hız için optimize edilmiştir.
 - 3. Çok çeşitli veri tiplerini destekler.

Redis

- Bu gereksinimleri 15 başlık altında toplayabiliriz (Devam):
 - 4. Anahtar tabanlı erişim.
 - 5. Birden fazla anahtar üzerinde çalışabilme.
 - 6. İşlemlerin atomikliği.
 - 7. Verilerin zaman aşımına uğraması.
 - 8. Tahliye durumları: RAM'in dolmaya başladığı durumlarda eski verilerden kurtulunması ve yeni veriye yer açılması.
 - 9. Akıllı önbellekleme.
 - 10. Dağıtık ağ sunucusu: Ağa bağlı uzak bir sunucu olarak kullanılabilir.
 - 11. İstekleri boru hattı (pipelining) mantığı ile iletme.
 - 12. Yüksek düzeyde erişilebilirlik.
 - 13. Veri sürdürülebilirliği (persistence).
 - 14. Ölçeklenebilirlik.
 - 15. Özel önbellek olarak kullanılabilme.

Memcached

- NoSQL mantığını uygulayan, verileri anahtar-değer ilişkisi şeklinde tutan önbellekleme sunucusudur.
- C ile yazılmıştır ve açık kaynak kodludur.
- Amaç veriyi önbellekte tutarak veriye erişimi ve dolayısıyla uygulamaları hızlandırmaktır.
- Genelde web uygulamalarında kullanılır.
- 2003 yılında LiveJournal isimli sosyal blog sitesinde kullanılmasıyla ortaya çıkmıştır.
- Sitenin popülerliğinin ve kullanıcı sayısının artmasıyla birlikte sitenin yavaşlığına dair şikayetlerde başlamıştır.
- Sitenin kurucusu Brad Fitzgerald (aynı zamanda Memcached in geliştiricisi) en büyük problemin veritabanından bilgi çekmek olduğunu belirtir.

Memcached

- İşleri hızlandırmak için LiveJournal'daki sık kullanılan nesneleri hafızada tutmak için sunucular kurulur.
- Bu nesneler renk, giriş bilgisi, stiller, yorumlar, site metni vs gibi bilgilerden oluşabiliyordu.
- Verilerin %95'i önbellekten getirilmeye başlandı ve sitedeki problemlerin çoğu ortadan kaldırıldı.
- Bugün Flickr, Slashdot, Wikipedia, Facebook gibi siteler bu tekniği uygulamaktadır.
- Verilere bellekten erişmek veritabanından erişmekten daha verimlidir ve uygulamaları daha ölçeklenebilir kılar.
- Verimliliği nasıl artırdığını bir örnekle inceleyelim.
- Bir etkinlik takvimi sunan bir web uygulaması oluşturacağınızı düşünün. Memcached yoksa, uygulama gelecek etkinlikler için gösterilecek bilgileri almak için veritabanına istekte bulunacak.

Memcached

- Veritabanı bilgileri diskten alır, web sunucusuna iletir, sunucu veriyi formatlar ve veriler web tarayıcısında gösterilir.
- Memcached olması durumunda ise veriler önce önbellekte aranır ve burada varsa bilgiler buradan aktarılır.
- Eğer memcached'de yoksa bilgiler veritabanından getirilir ve aynı zamanda bu bilgiler bir sonraki istekler düşünülerek önbelleğe aktarılır.
- Burada ek bir işlem yapıldığı düşünülebilir. Ancak diskten okuma ve bellekten okuma maliyetleri göz önünde bulundurulduğunda, bellekten okumayla web sunucusunun daha etikili ve hızlı bir hizmet sunacağı görülecektir.
- Göz önünde bulundurulması gereken bir nokta Memcached'in sürekli depolama imkanı verecek bir yapı olmamasıdır.

Memcached

- Memcached'i sorgulayarak tuttuğu tüm bilgileri getiremezsiniz ya da Memcached'deki tüm bilgileri diske yazamazsınız.
- Bir bilginin memcached'de olup olmadığını anlamanız için sunucuyu sorgulamanız gerekir.
- Çünkü memcached sürekli depolama hizmeti veren bir birim değildir. Optimize edilmiş bir önbellekleme sunucusudur.
- Memcached'in kendi içinde bir güvenlik mekanizması yoktur.
- Ancak bazı teknikler uygulayarak güvenliği sağlayabilirsiniz.
- Memcached sunucusu çökerse bilgilerde gidecektir.
- Ancak bu bir problem teşkil etmez. Çünkü önbelleğe alınan bilgilerin kaynağı hala veritabanında durmaktadır.
- Yazdığınız kod sadece önbellekteki veriyi bulamayacak ve dolayısıyla veri veritabanından çekilecektir.

Memcached

- Memcached'in sunucu ve istemci olmak üzere iki bileşeni vardır.
- Sunucu verilerin anahtar-değer ilişkisi şeklinde tutulmasından ve verinin anahtar değer kullanılarak getirilmesinden sorumludur.
- Hızlı ve basittir ancak 2 kısıtlaması vardır.
 - Anahtarlar 250 karakteri geçemez.
 - Depolayabileceğiniz veri parçasının boyutu 1MB'yi aşamaz.
- Her memcached sunucusu atomiktir. Yani diğer sunucuların ne durumda olduğunu bilmez ve önemsemez.
- Hem Linux hem de Windows işletim sistemi için desteği vardır.
- Yüksek veritabanı yüklerine sahip web siteleri için oldukça verimli ve daha hızlıdır.
- Sunucuyla iletişim TCP ve UDP protokolleri aracılığı ile olur.

Memcached

- Veriyi depolamak ve veriye ulaşmak için 4 adet basit komutları vardır.
 - **SET:** Memcached'e yeni veri ekler ya da varolan veriyi yeni veri ile değiştirir.
 - **ADD:** Sadece anahtar yoksa veriyi ekler. Anahtar varsa NOT_STORED döndürür.
 - **REPLACE:** Sadece anahtar varsa veriyi ekler. Anahtar yoksa NOT_STORED döndürür.
 - **GET:** Veriyi getirir.
- Veriyi silmek için **DELETE ANAHTAR** komutu kullanılır. Bu komut kullanarak veri silindiğinde ya da verinin süresi dolduğunda aslında veri silinmez. Veri hafıza durur ancak anahtar geçersiz kılınır. Eğer bellek ihtiyacı duyulursa hafızadan silinir.
- Veri depolanırken kullanılabilecek bilgiler:
 - **keyname:** Veriye ulaşmak için kullanılacak anahtarın adı
 - **flags:** Veri dışında ek bilgiler tutmak için kullanılır.
 - **expiration time:** Verinin ne kadar süre tutulması gerektiğini söyler. 0 hiç silme anlamına gelir.
 - **bytes:** Hangi boyutta veri tutulacağını ayarlar.

Memcached

- Memcached'in kendi içinde bir güvenlik mekanizması olmadığı için sunucuya erişebilen herkes verilere de erişebilir.
- Dolayısıyla memcached sunucularını dış dünyaya açmamanız gerekir. En azından memcached'in kullandığı port dışarıya erişime kapatılmalıdır.
- Ek bir güvenlik mekanizması olarak anahtarlarınızı karıştırabilirsiniz (obfuscate). Bu da sunucuya ulaşılsa bile veriye erişimi, neyin nasıl tutulduğunu anlamayı zorlaştıracaktır.
- Memcached'de neyin önbellekleneceğine nasıl karar verilecek?
 - Genelde kullanıcı şikayetlerine neden olan kısımlar veritabanı işlemleridir. Dolayısıyla en sık ve en uzun süre çalışan sorgular belirlenmelidir. Buradan gelen sonuçlar önbelleğe alınabilir.
 - Tüm HTML sayfasını önbelleğe almanın mantığı yoktur. Onun yerine sayfanın çoğu bölümünde aynı şekilde kullanılan HTML parçalarını ve içindeki verileri önbelleğe alabilirsiniz.

Redis vs. Memcached

- Her ikisi de NoSQL veritabanları kategorisinde yer alır.
- Her ikisi de anahtar-değer veri modeline sahiptirler.
- Her ikisi de veriyi RAM'de tutar.
- İkisi de oldukça hızlıdır.
- Her ikisinin de önbelleklenmesi alanında oldukça olgun ve popüler teknolojilerdir.
- Redis buz dağı ise Memcached buz dağının sadece görünen kısmıdır.

Memcached Ne Zaman Kullanılabilir?

- Oldukça küçük ve statik verinin önbelleklenmesinde kullanılabilir. HTML kod parçacıkları gibi.
- Basit senaryolarda daha etkilidir.
- Özellikle metadataların tutulmasında daha az hafıza kullanır.
- Sadece okuma amaçlı kullanılacak String türündeki verilerin tutulması için idealdir.
- Çok işparçacıklı yapıya sahip olması Redis'e göre ölçeklenebilirliğini artırır.
- Dolayısıyla kendine daha fazla kaynak ayırabilir.
- Redis'in tek iş parçacıklı yapısı yataya kümeler (cluster) halinde genişleyebilmesine izin verir.
- Ancak kümeleme daha karmaşık ve maliyetli bir iştir.
- Memcached'in kopya oluşturma özelliği yoktur.
- Memcached yüksek trafiğe sahip sitelerde daha iyi çalışır.
- Tek seferde bir çok bilgi okuyabilir ve cevap verme süresi oldukça iyidir.

Redis Ne Zaman Kullanılabilir?

- Uygulamalar 512 MB büyüklüğünde anahtarlar ve değerler içeren büyük veriler kullanmak zorunda olabilir.
- Bu durumda Redis tek bir kayıt için yaklaşık 1GB'ye kadar verinin önbelleklemesini destekleyecek kapasitededir.
- Kopya oluşturmayı destekler.
- Kopya oluşturma işlemi de genelde master-slave düğümler oluşturarak sağlanır.
- Slave düğümler master düğümü dinler.
- Master güncellendiğinde slave düğümler otomatik güncellenir.
- Master düğüm slave düğümleri asenkron olarak da güncelleyebilir.

Redis Ne Zaman Kullanılabilir?

- Redis'in C, Ruby, Java, Javascript, Python gibi diller için API'si mevcuttur.
- Redis'in yayınla/üye ol mesajlaşma yapısını ve seçtiğiniz programlama dilini kullanarak yüksek performanslı mesajlaşma uygulamaları geliştirebilirsiniz.
- Bazen çok kısa bir süre içinde milyonlarca parçadan oluşan verileri önbellekleme ihtiyacı duyabilirsiniz. Bunu Redis ile gerçekleştirebilirsiniz.
- Redis az miktarda bellek kullanır. Dolayısıyla Raspberry Pi gibi cihazlarda IOT tabanlı uygulamalar geliştirmek için Redis daha uygundur.
- Redis işlemleri (transaction) destekler. Yani komutları tek seferde çalıştırmak yerine kuyruk halinde işleyebilir.

Redis vs. Memcached

	Memcached	Redis
Milisaniye altında gecikmeler	Evet	Evet
Geliştiriciler için kullanım kolaylığı	Evet	Evet
Veri bölütleme	Evet	Evet
Farklı programlama dillerine verdiği destek	Evet	Evet
Gelişmiş veri yapıları	-	Evet
Çok işparçacıklı mimari	Evet	-
Anlık görüntü alma	-	Evet
Kopya oluşturma	-	Evet
İşlemler	-	Evet
Yayınla/Üye ol tarzı mesajlaşma	-	Evet
Lua skript desteği	-	Evet
Coğrafi veri desteği	-	Evet

Redis vs. Memcached Hafıza Kullanımı

Ölçek	Hafıza Kullanımı (MB)	
	Memcached	Redis
Tek Düğüm	10,876	14,740
3 Düğümlü Küme	3,630	4958

Memcached % 6.6 ek hafıza kullanır.

Redis % 44.5 ek hafıza kullanır.

Kaynak: Comparative Evaluation of the Performance of In-Memory Key-Value Stores: Redis vs Memcached

Redis vs. Memcached Çalışma Yüğü

Çalışma Yüğü	Aynı Anda Çok Fazla İstemci		Aynı Anda Az Sayıda İstemci	
	Tek Düğüm	Küme	Tek Düğüm	Küme
Yük-1 (% 500-% 50G)	Memcached	Memcached	Memcached	Memcached
Yük-2 (% 950-% 5G)	Memcached	Memcached	Memcached	Memcached
Yük-3 (% 50-% 95G)	Redis	Redis	Redis	Redis
Yük-4 (% 900-% 10E)	Redis	Redis	Redis	Redis
Yük-5 (% 100-% 90E)	Redis	Redis	Memcached	Memcached

O: Okuma, G: Güncelleme, E: Ekleme

Yeterli belleğiniz yoksa Memcached kullanın

Comparative Evaluation of the Performance of In-Memory Key-Value Stores: Redis vs Memcached

Kimler Redis Kullanıyor?



Uber



Airbnb



Pinterest



Instagram



Shopify



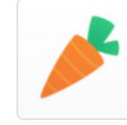
Twitter



Udemy



Slack



Instacart



Robinhood



reddit



Lyft



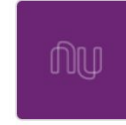
StackShare



Delivery Hero



LaunchDarkly



Nubank



Stack



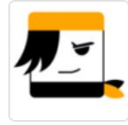
Tokopedia



medium.com



Bitpanda



Alibaba
Travels



Accenture



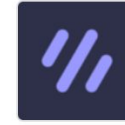
HENNGE K.K.



Microsoft



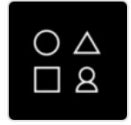
CircleCI



Graphy



deleokorea



Figma



trivago



CRED



Stack
Overflow



Kmong



Hepsiburada



ViaVarejo



Asana



N26



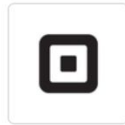
DoorDash



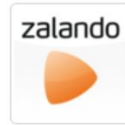
Ruangguru



Typeform



Square



Zalando



BirdView



KAVAK

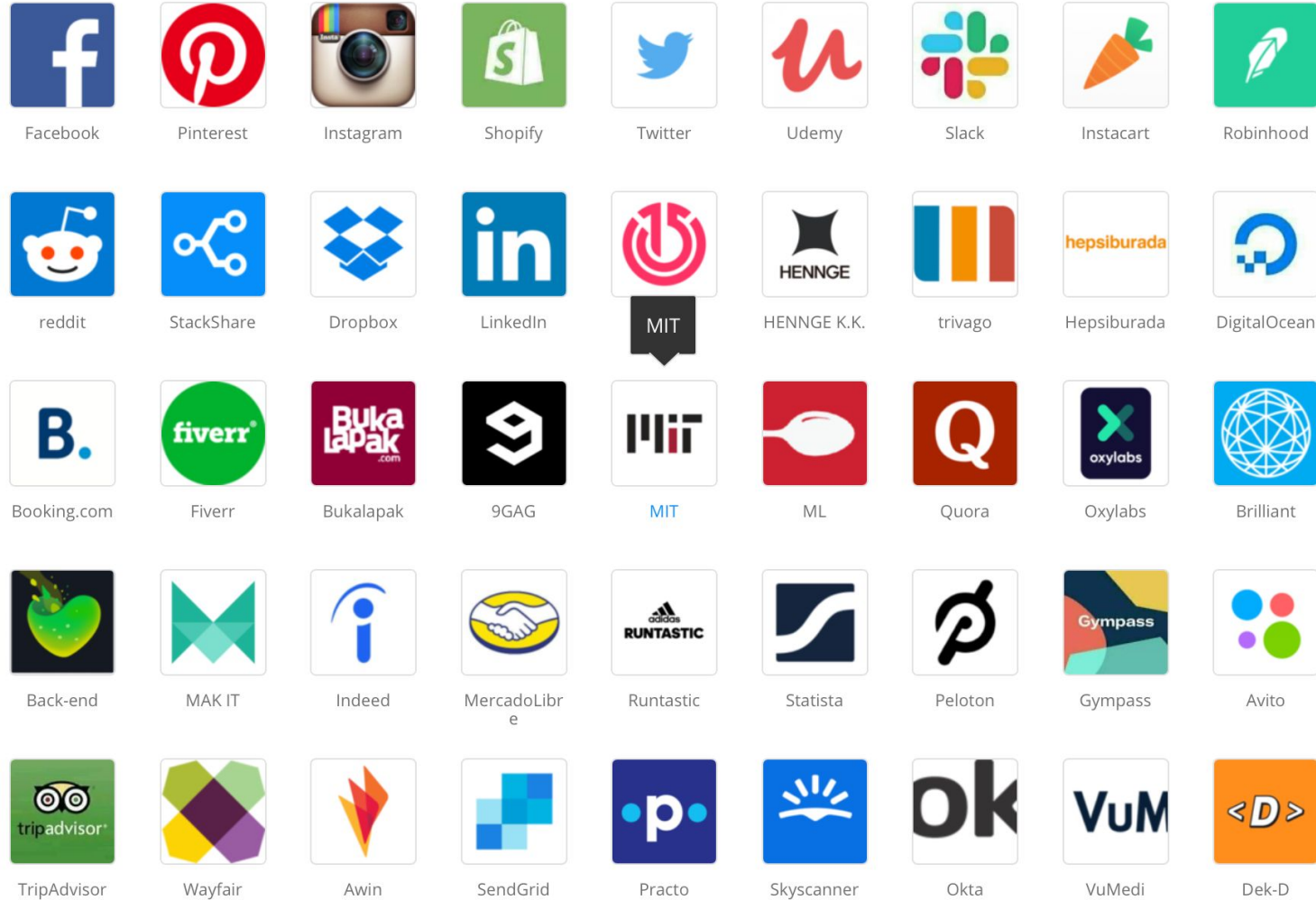


Durstexpress
GmbH



Glovo

Kimler Memcached Kullanıyor?



Redis ve Memcached Kullanım Örnekleri

- Twitter'da kullanıcıların akışı (timeline) Redis'te tutuluyor. Kullanılan veri tipi ise List. Bir kullanıcının son 800 tweeti Redis'te tutuluyor.
 - Kaynak: <https://www.infoq.com/presentations/Twitter-Timeline-Scalability/>
- Twitter en son ve sık erişilen tweetleri tutmak için Twemcache kullanıyor. Twemcache Twitter tarafından memcached üzerine geliştirilmiş.
- Pinterest ise takipçilerin, takip edilenlerin listesini, panolarınızı takip eden kişilerin listesini, takip ettiğiniz panoların listesini Redis'te tutuyor.
- The Walking Dead: Road to Survival tarzı oyunlar yapan Scopely firması liderlik sıralaması, API yönetimi, kuyruk iş yükü yönetimi gibi işlerde Redis kullanıyor.
- Ünlü giyim şirketi GAP e-ticaret sitelerinde kullanıcıların sepete ekledikleri her ürün için anlık kargolama bilgisini göstermek amacıyla Redis'i tercih etmiştir.
- Facebook memcache'i veritabanı yükünü azaltmak için kullanıyor.

Memcached: NodeJS Uygulaması

- OSX için yükleme:
 - **brew install memcached**
- Linux için yükleme:
 - **sudo apt-get install memcached**
- OSX ile terminalde Memcached'i başlatma:
 - **memcached -l 127.0.0.1 -p 11211 -vv**
- Linux için detaylı kurulum:
<https://www.cyberciti.biz/faq/install-and-configure-memcached-on-ubuntu-linux18-04/>
- Windows için detaylı kurulum:
<https://commaster.net/posts/installing-memcached-windows/>
- Memcached Docker İmajı: https://hub.docker.com/_/memcached
- Basit NodeJS Uygulaması

Redis: NodeJS Uygulaması

- OSX için yükleme:
 - **brew install redis**
- Linux için yükleme:
 - **sudo apt-get install redis-server**
- OSX ile terminalde Redis'i başlatma:
 - **redis-server**
- Linux için system açıldığında Redis'i başlatma:
 - **sudo systemctl enable redis-server.service**
- Windows için detaylı kurulum:
 - <https://redislabs.com/blog/redis-on-windows-8-1-and-previous-versions/>
 - <https://redislabs.com/blog/redis-on-windows-10/>
- Redis Docker İmajı: https://hub.docker.com/_/redis/
- [Basit NodeJS uygulaması](#)
- Daha Kapsamlı NodeJS Uygulaması:
<http://www.borakasmer.com/nodejs-uzerinde-redis-kullanimi-ve-refactoring/>

Redis: JWT Kara Listeleme

- Twitter kullandığınızı hayal edin. Oturum açın, bir tweet yazın, sonra başka birinin tweetini retweetleyin.
- Her eylem için, söz konusu eylemi gerçekleştirmeden önce kimliğinizin doğrulanması ve yetkilendirilmesi gerekir.
- Geleneksel yaklaşımda işler şu şekilde ilerler:
 - Kullanıcı adınız ve şifreniz ile giriş yaparsınız.
 - Sunucu önce kullanıcının kimliğini doğrular.
 - Sunucu daha sonra bir oturum jetonu oluşturur ve bu jeton, kullanıcının bilgileriyle birlikte bir veritabanında saklar. (Not: Bir oturum belirteci, şuna benzer: fsaf12312dfsdf364351312srw12312312dasd1et3423r)
 - Sunucu daha sonra size frontend mobil veya web uygulamasına bir oturum jetonu gönderir.

Redis: JWT Kara Listeleme

- Geleneksel yaklaşımda işler şu şekilde ilerler (devam):
 - Sonra bir tweet yazıp gönderdiğiniz varsayalım. Ardından, sunucunun kim olduğunuzu tanımlayabilmesi için tweet'inizle birlikte uygulamanız oturum jetonunu da (bir tanımlama bilgisi veya bir başlık aracılığıyla) gönderir.
 - Sunucu oturum jetonunu aldığı anda, kullanıcının kim olduğunu bilmeyecektir, bu nedenle, o jetondan gerçek kullanıcının bilgilerini (userId gibi) almak için bunu veritabanına gönderir.
 - Kullanıcı varsa ve bu işlemi yapmasına (yani bir tweet göndermesine) izin varsa, sunucu işlemi yapmasına izin verir.
 - Ve son olarak, frontende tweet'in gönderildiğini söyler.
- Burada veritabanına gidip jetonun kime ait olduğu bilgisini sorgulamak yavaş bir iştir ve kullanıcının yaptığı her işlem için tekrarlanması gerekir.
- Dolayısıyla, her bir API çağrısı, genel yanıt süresini yavaşlatabilen en az iki yavaş DB çağrısına yol açar.

Redis: JWT Kara Listeleme

- Bu problem iki şekilde çözülebilir:
 - 1. Bir şekilde kullanıcılar için veritabanı aramasını tamamen ortadan kaldırmak
 - 2. Ek atlamamanın önemli olmaması için ekstra veritabanı aramasını çok daha hızlı yapmak.
- 1. Yöntem için farklı yollar mevcuttur:
 - Durumu sunucunun belleğinde saklayabilirsiniz. Ancak bu durum yalnızca belirli bir sunucuda mevcut olduğundan, ölçeklendirdiğinizde sorunlara neden olur.
 - "Yapışkan oturumlar/sticky session" kullanabilirsiniz. Burada, yük dengeleyiciye, siz ölçeği büyüttükten sonra bile trafiği her zaman belirli bir sunucuya yönlendirmesini söylersiniz. Yine bu, farklı ölçeklendirme sorunlarına neden olur ve sunucu küçülürse (ölçek küçülür), tüm durumu kaybedersiniz.
 - JWT kullanmak.

Redis: JWT Kara Listeleme

- Şimdi JWT kullanımını inceleyelim.
- JWT, özellikle oturum olarak kullanıldığında, veritabanı aramasını tamamen ortadan kaldırarak sorunu çözmeye çalışır.
- Ana fikir, kullanıcının bilgilerini oturum jetonunun kendisinde depolamaktır.
- Bu nedenle, bazı uzun rasgele dizeler yerine, gerçek kullanıcı bilgilerini oturum jetonunun kendisinde saklanır.
- Güvenliğini sağlamak için, yalnızca sunucu tarafından bilinen bir sır kullanarak jeton parçasının imzalanmasını sağlanır.
- Bu nedenle, istemci ve sunucu, jetonun kullanıcı bilgisi kısmını görebilse de, ikinci kısım, yani imzalanan kısım, yalnızca sunucu tarafından doğrulanabilir.

Redis: JWT Kara Listeleme

Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.TuiHzLTBa-K_igyZLw2-SCdjJtIs-x4S0KsEXhsKSTU
```

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  your-256-sdfdsfdfsdf
) ☐ secret base64 encoded
```

Jetonun pembe bölümü payload'a (kullanıcı bilgisi) sahiptir ve hem istemci hem de sunucu tarafından görülebilir. Ancak mavi kısım gizli bir string ile, başlık ve yükün kendisi kullanılarak imzalanır. Dolayısıyla, kullanıcı yükü kurcalarsa (farklı bir kullanıcının kimliğine bürünürse), imza farklı olacak ve kimliği doğrulanmayacaktır.

Redis: JWT Kara Listeleme

- JWT ile kimlik doğrulama aşamaları:
 - Kullanıcı adı ve şifre ile giriş yapılır. Sunucu, veritabanını sorgulayarak kullanıcının kimliğini doğrular.
 - Sunucu daha sonra kullanıcının bilgilerini ve sırrını kullanarak bir JWT oturum jetonu oluşturur (veritabanı dahil değildir).
 - Sunucu daha sonra frontend uygulamaya bir JWT gönderir. Gelecekteki işlemler için, kullanıcı her seferinde oturum açmak yerine kullanıcıyı tanımlayan JWT'yi gönderebilir.
 - Bir JWT şuna benzer: <header>.<payload>.<signature>
 - Ardından, bir tweet yazıp gönderdiğiniz varsayalım.
 - Tweet'inizin metniyle birlikte gönderdiğinizde, sunucunun kim olduğunuzu tanımlayabilmesi için uygulamanız JWT'yi de (bir tanımlama bilgisi veya bir başlık aracılığıyla) gönderir.
 - Sunucu JWT'yi aldığı anda, imzalı kısmı doğrulamak için sırrı kullanır ve yük kısmından kullanıcı bilgilerini alır. Böylece DB çağrısını ortadan kaldırılır.
 - İmza kontrol edilir, kontrol başarılı ise eylemi yapmalarına izin verir. Sonunda tweet'in kaydedildiği frontende bildirilir.
 - Her kullanıcı eylemi için sunucu yalnızca imzalanan kısmı doğrular, kullanıcı bilgilerini alır ve kullanıcının bu eylemi yapmasına izin verir. Böylece DB çağrısını tamamen atlanır.

Redis: JWT Kara Listeleme

- Ancak JWT hakkında bilinmesi gereken ek ve önemli bir şey daha var.
- JWT kendisini sona erdirmek için bir son kullanma süresi kullanır. Genellikle 5 dakika ile 30 dakika arasında ayarlanır.
- Bağımsız olduğu için kolayca iptal edilemez/geçersiz kılınamaz/güncellenemez. Sorunun özü aslında burada yatmaktadır.
- JWT ile ilgili en büyük sorun, jeton iptal sorunudur. Süresi dolana kadar çalışmaya devam ettiği için sunucunun onu iptal etmesinin kolay bir yolu yoktur.
- **Oturum kapatma, gerçekten oturumunuzu kapatmaz!**
 - Tweet attıktan sonra Twitter'dan çıkış yaptığınızı düşünün. Sunucudan çıkış yaptığınızı düşünebilirsiniz, ancak durum böyle değil. Çünkü JWT bağımsızdır ve süresi dolana kadar çalışmaya devam edecektir. Bu, 5 dakika veya 30 dakika veya belirtecin bir parçası olarak ayarlanan süre ne olursa olsun olabilir. Dolayısıyla, birisi bu süre içinde o jetona erişim elde ederse, süresi dolana kadar erişime devam edebilir.

Redis: JWT Kara Listeleme

- Kullanıcıları engellemek, onları hemen engellemez!
 - Twitter'ın veya gerçek kullanıcıların sistemi kullandığı bazı çevrimiçi gerçek zamanlı oyunların moderatörü olduğunuzu hayal edin. Ve bir moderatör olarak, birisinin sistemi kötüye kullanmasını hızlı bir şekilde engellemek istiyorsunuz. Yine aynı sebepten bu işlemi yapamazsın. Engelledikten sonra bile, belirtecin süresi dolana kadar kullanıcı sunucuya erişmeye devam edecektir.
- Eski veriler olabilir!
 - Kullanıcının bir yönetici olduğunu ve daha az izne sahip normal bir kullanıcıya indirgendiğini hayal edin. Yine, bu işlem hemen gerçekleşmeyecek ve kullanıcı, belirtecin süresi dolana kadar yönetici olmaya devam edecektir.
- JWT'ler genellikle şifrelenmez, bu nedenle ortadaki adam saldırısı gerçekleştirebilen ve JWT'yi ele geçiren herkes artık kimlik doğrulama bilgilerinize sahiptir. MITM saldırısının yalnızca sunucu ile istemci arasındaki bağlantıda tamamlanması gerektiği için bu daha kolaydır.

Redis: JWT Kara Listeleme

- Her defasında JWT'nin durumsuz/statless olduğundan bahsederiz.
- Gerçek dünyadaki birçok uygulamada, sunucuların kullanıcının IP'sini tutması ve hız sınırlaması ve IP beyaz listesi için API'leri izlemesi gerekir.
- Bu nedenle, son derece hızlı bir veritabanı kullanmanız gerekecek. Bir şekilde uygulamanızın JWT ile durumsuz hale geldiğini düşünmek gerçekçi değildir.
- Popüler Çözüm:
 - İptal edilen tokenların bir listesini bir veritabanında saklamak ve her arama için kontrol etmek.
 - Token, iptal edilenler listesinin bir parçasıysa, kullanıcının bir sonraki eylemi gerçekleştirmesi engellenir.
 - Ancak token iptal edilip edilmediğini kontrol etmek için DB'ye bir ekstra çağrı yapmanız gerekir ve bu nedenle JWT'yi amacı dışında kullanmış olursunuz.

Redis: JWT Kara Listeleme

- **Sonuç:**

- JWT, veritabanı aramasını ortadan kaldırmasına rağmen, bunu yaparken güvenlik sorunları ve diğer karmaşıklıkları ortaya çıkarır. Güvenlik ikilidir: ya güvenlidir ya da değildir. Bu nedenle, kullanıcı oturumları için JWT kullanmayı tehlikeli hale getirir.

- **Nerede kullanabilirim:**

- Backend'de sunucudan sunucuya (veya mikro hizmetten mikro hizmete) iletişim yaptığınız gereken ve bir hizmetin yetkilendirme amacıyla farklı bir hizmete göndermek için bir JWT oluşturabileceği senaryolar vardır.
- Kullanıcının e-postasını doğrulamak için bir JWT tek seferlik kısa ömürlü bir token olarak gönderebileceğiniz parola sıfırlama gibi diğer kullanım alanları da vardır.

Redis: JWT Kara Listeleme

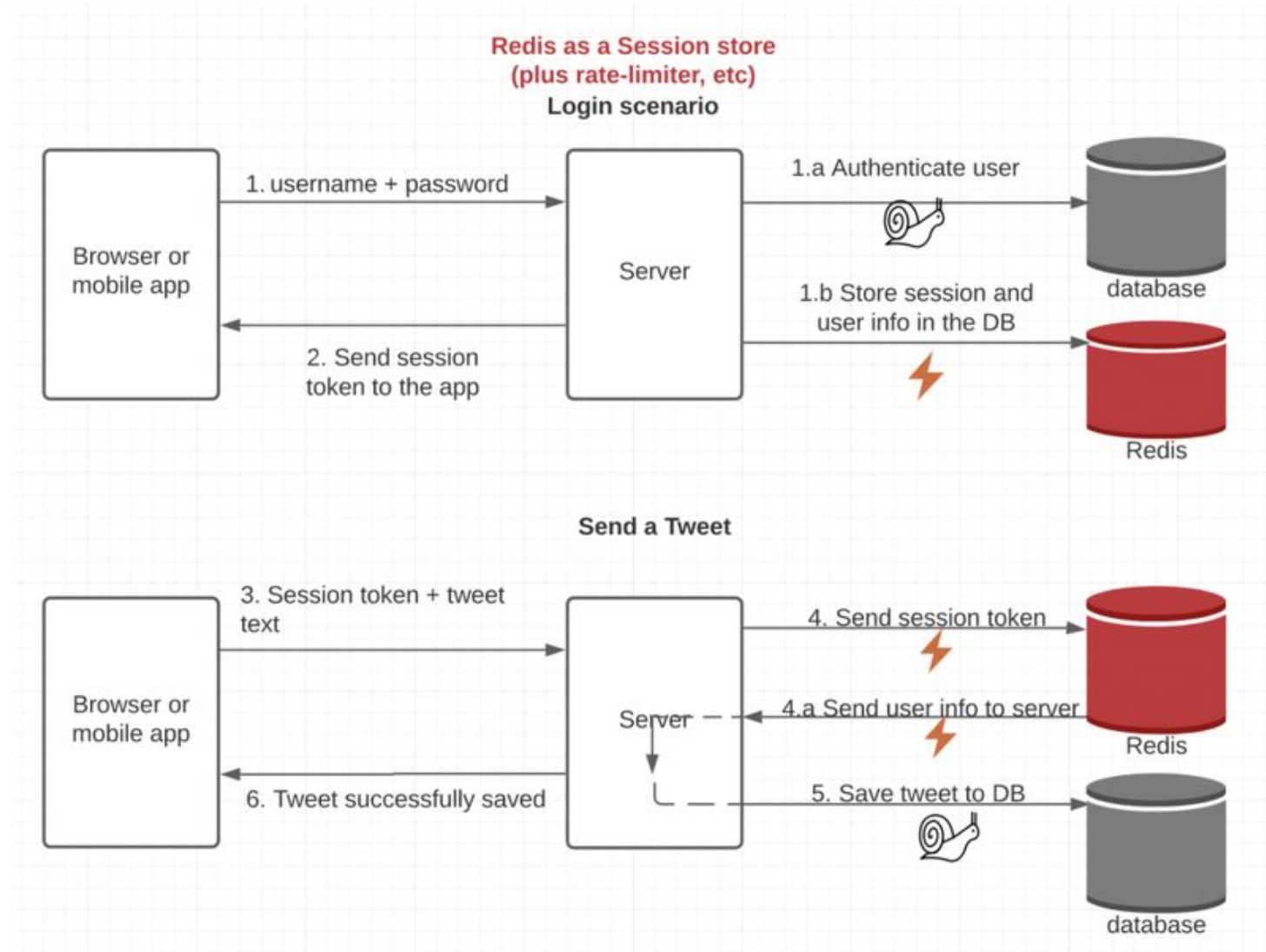
- **Peki JWT kullanamıyorsam ne yapmalıyım?**

- Çözüm, JWT'yi oturum amacıyla hiç kullanmamaktır. Ancak bunun yerine, geleneksel, savaş alanında test edilmiş/battle-tested yolu daha verimli bir şekilde işletmek.
- Yani veritabanı aramasını o kadar hızlı (milisaniyenin altında) yapın ki, ek çağrı önemli olmasın.

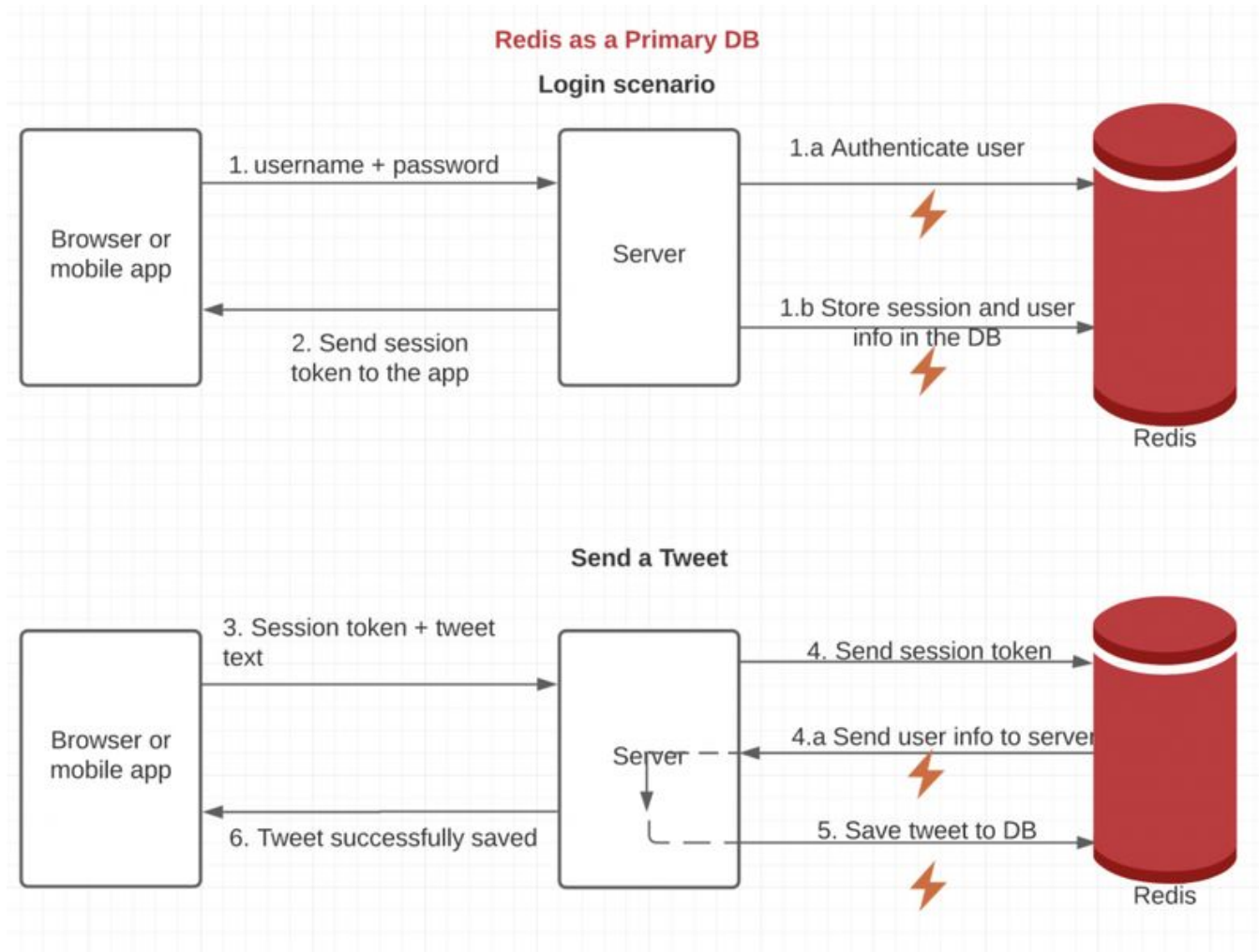
- **Aramayı, önemli olmayacak kadar hızlı yapmak (savaş alanında test edilmiş bir çözüm):**

- Milisaniyelerin altında milyonlarca isteğe hizmet verebilecek kadar hızlı yapan herhangi bir veritabanı var mı?
- Elbette var. **Redis:** Her gün milyarlarca kullanıcıya hizmet veren binlerce şirket tam da bu amaç için Redis kullanıyor!
- Redis'i bir oturum/ssesyon deposu olarak ve diğer veritabanlarını birincil veritabanı olarak kullanmanız yeterlidir.
- Redis'i tüm veri katmanınız için birincil veritabanı olarak da kullanabilirsiniz. Bu senaryoda, mimariniz çok daha basit hale gelir ve temel olarak her şey çok hızlı hale gelir.

Redis: JWT Kara Listeleme



Redis: JWT Kara Listeleme



Redis: JWT Kara Listeleme

- **Peki JWT kullanmak istiyorsam ne yapmalıyım?**
 - Çözüm: Redis'i Tokenları kara listeye almak için kullanmak.
- JWT'de en büyük problem bir kullanıcının erişiminin nasıl iptal edileceğidir.
- Elbette, JWT'lerin bir son kullanma süresi vardır ve istediğiniz kadar kısa ömürlü olabilir. Ancak erişim belirtecinin süresi dolduğunda, JWT geçersizdir ve istemcinin sunucunuzla yeniden kimlik doğrulaması yapması gerekir. Bu da tabii ki kullanıcı deneyimine olumsuz etki eder.
- Ancak kullanıcının kasıtlı olarak sisteminizden çıkış yaptığını varsayalım? Ya da güvenliğiniz tehlikeye düştüğünden ve korktuğunuz için onları engellemek mi istiyorsunuz?
- Yapamazsınız çünkü hâlâ süresi dolmamış bir tokenı varsa, hâlâ erişim sağlayabilirler.

Redis: JWT Kara Listeleme

- Bu sorunun bir çözümü var, ancak bazı ekstra işlemler yapılması gerekir. Bir JWT kara listesi/reddetme listesi tutmayı gerektiren bir tekniktir.
- Bir JWT kara listesi/reddetme listesi, artık sisteminize erişim vermemesi gereken tokenların bir listesidir.
- Bu listeyi nerede tutacağınız size kalmıştır. Geleneksel bir veritabanı kullanabilirsiniz.
- Ancak Redis gibi bir bellek içi veri önbelleği kullanmak çok daha iyi bir yaklaşımdır.
- Bellek içi veri önbelleği, diskte depolanan verilere göre çok daha hızlı ve daha öngörülebilir arama süreleri sağlar.
- Şimdi NodeJS, ExpressJS ve Redis kullanarak kara listeleme işlemini nasıl yapacağımızı görelim.

Redis: JWT Kara Listeleme

- JWT Kara Listeleme İçin Adımlar:
 - Öncelikle, etkin JWT'lerin bir listesini tutabilmeniz için Express sunucu uygulamanızı başlatmanız ve Redis'i kurmanız gerekir.
 - Bir JWT'nin depolanmasını ve sonrasında iptalini test etmek için, bir kullanıcı oluşturmanız ve bu kullanıcı için bir JWT oluşturmanın bir yoluna ihtiyacınız olacak.
 - Kullanıcı uygulamanızla herhangi bir şey yapmaya çalıştığında, o JWT ile kullanıcının kimliğini doğrulamalısınız.
 - Bu üç aşamalı bir süreçtir. Öncelikle, JWT'nin sağlandığından emin olmalısınız.
 - Ardından, sağlanan JWT'nin Redis'te depolanan kara liste/reddetme listesinde olup olmadığını test etmelisiniz.
 - Değilse, sonunda JWT'nin geçerli olup olmadığını doğrulamalısınız.
 - Kullanıcı oluşturduğunuz güvenli rotayı ziyaret etmeye çalıştığında, sağladıkları JWT önce kara liste/reddetme listesine göre kontrol edilmeli ve ardından erişim verilmeden önce doğrulanmalıdır.
 - Her şey yolunda gitse bile şu anda bir JWT'yi iptal etmenin bir yolu yoktur, bu nedenle kara listede/reddetme listesinde hiçbir şey yoktur.

Redis: JWT Kara Listeleme

- JWT Kara Listeleme İçin Adımlar (Devam):
 - Bir kullanıcının oturumu kapatmasını sağlamak için bir rota oluşturulmalıdır.
 - Bu rotaya ulaşıldığında, kullanıcının JWT'si Redis'te tutulmalıdır.
 - Redis'i çok sayıda süresi dolmuş jetonla doldurmamak için anahtar, tokenın süresi dolduğunda sona erecek şekilde ayarlanmalıdır.
 - Artık kullanıcının JWT'si teknik olarak hala geçerlidir (süresi dolana kadar), ancak kara listede/reddetme listede olduğu için sonraki isteklerde durdurulacaktır.
 - Ancak, oturumu kapatmamış ve tokenı hâlâ geçerli olan bir kullanıcının erişimini reddetmek istiyorsanız bu bir çözüm olmayacaktır.
 - Çünkü o sırada kullanıcının erişim tokenın ne olduğunu bilemezsiniz ve bu nedenle ön belleğe ne koyacağınızı bilemeyeceksiniz.
 - Bu sorunu çözmek için de, bir kullanıcının kullanıcı kimliğini redis yoluyla kara listeye alma ve JWT doğrulaması sırasında bu kara listeyi de kontrol etme özelliğini eklemelisiniz. JWT kara listesinden farklı olarak, bu girişin kendisiyle ilişkilendirilmiş bir son kullanma süresi olmayacaktır.
- Örnek Kod

Redis: JWT Kara Listeleme

- Peki bu yöntem iyi bir yöntem midir? Hayır.
- JWT'lerin en büyük avantajı oturum doğrulamasını hızlı yapmalarıdır. Bir kara liste/reddetme listesi tutuyorsanız ve bunu her API çağrısında sorgulamanız gerekiyorsa bu avantajı kaybedersiniz.
- Bunun yerine, hem JWT'lerin hem de opak tokenların güçlü yönlerini birleştiren bir oturum yönetimi çözüm olabilir.
- JWT'nin süresi dolduğunda, istemci yeni bir JWT ve yeni bir opak yenileme tokenı istemek için opak yenileme tokenı kullanır. Bu işlem, yenileme tokenı döndürme (refresh token rotation) olarak bilinir.
- Bu yöntemle kullanıcı oturum açtığında, sunucu hem kısa ömürlü bir JWT (erişim tokenı) hem de uzun ömürlü opak bir token (yenileme jetonu) üretilir.
- [Refresh Token ReactJS İçin Detaylı Örnek](#)

Redis: JWT Kara Listeleme

- Bir kullanıcıya erişim verildiğinde, bu jetonların her ikisi de istemciye gönderilir.
- Kullanıcı bir kaynağa erişmeye çalıştığında, her istekle birlikte JWT erişim tokenı gönderir.
- **Peki iki farklı token ve yenileme tokenı döndürmenin avantajları ve dezavantajları nelerdir?**
 - Erişimi iptal etmek istiyorsanız, yapmanız gereken tek şeyin sunucu tarafında opak tokenı geçersiz kılmak olmasıdır.
 - Ardından, yenileme uç noktası çağrıldığında, sunucu opak tokenı arar, süresinin dolduğunu görür ve kullanıcının oturumunu kapatır.
 - İstemcide hala geçerli olan JWT'ler sorunu hala çözümsüzdür.
 - Ancak, bu JWT'leri çok daha kısa ömürlü hale getirebileceğiniz için (sadece birkaç dakika bile), çok büyük bir sorun olmayacaktır.

Örnek Redis Kullanımları

- **Sepet Yönetimi:** Kullanıcı sepete ürün eklediğinde veriler Redis'te tutulur. DB'ye yazmadan hızlı şekilde sepet güncellenir.
- **Oturum (Session) Saklama:** Kullanıcı giriş yaptıktan sonra session bilgisi Redis'e kaydedilir. Sunucu restart olsa bile oturum devam edebilir.
- **Rate Limiting (Giriş deneme limiti):** Bir IP'nin kaç kez login denediği Redis'te sayılır. Limit aşılsa geçici olarak engellenir.
- **Stok Geçici Rezervasyon:** Ürün sepete eklenince Redis'te geçici stok düşülür. Ödeme yapılmazsa stok geri iade edilir.
- **Bildirim Kuyruğu:** Kayıt, sipariş gibi işlemler Redis kuyruğa atılır. Arka planda bu veriler işlenip email/SMS gönderilir.
- **Online Kullanıcı Listesi:** Giriş yapan kullanıcılar Redis set içinde tutulur. Kimlerin aktif olduğu anlık gösterilebilir.
- **Önbellek (Ürün listeleme):** Sık kullanılan ürün listeleri Redis'te cache'lenir. Veritabanı yükü azaltılır ve hız artar.
- **Sipariş durum takibi:** Sipariş "hazırlanıyor / kargoda / teslim edildi" gibi durumları Redis'te tutulur. Durum değişiklikleri anlık frontend'e yansır.

Örnek Redis Kullanımları

- **Sipariş durum takibi:** Sipariş “hazırlanıyor / kargoda / teslim edildi” gibi durumları Redis’te tutulur. Durum değişiklikleri anlık frontend’e yansır.
- **API sonuçlarını önbellekleme:** Sık kullanılan API response’ları Redis’te saklanır. Aynı request gelirse direkt Redis’ten döner.
- **Ürün öneri sistemi :** “Bunları da beğenebilirsiniz” algoritma sonuçları Redis’te tutulur. Her kullanıcı için hızlı öneri üretilir.
- **Kullanıcı aktivite takibi:** Kullanıcının son yaptığı işlemler Redis listesinde tutulur. “Son görüntülenen ürünler” gibi özellik yapılır.
- **Lokasyon bazlı aktif kullanıcılar:** Belirli şehir/ülkeye göre online kullanıcı sayısı Redis set ile tutulur. Harita veya analiz ekranı oluşturulur.
- **Sık sorgu optimizasyonu:** Admin panelindeki istatistikler Redis’te cache’lenir. Her istek DB’ye gitmez, sadece belirli aralıklarla güncellenir.
- **Gerçek zamanlı chat mesajlaşma:** Mesajlar Redis pub/sub ile anlık olarak diğer kullanıcılara iletilir. Veri kalıcı DB’ye ayrıca yazılır, Redis sadece iletim için kullanılır.

Kaynaklar

- <https://memcached.org>
- <https://redis.io>
- <https://stackshare.io/memcached>
- <https://stackshare.io/redis>
- <https://techstacks.io/tech/redis>
- <https://techstacks.io/tech/memcached>
- [Windows için Memcached: https://github.com/jefyt/memcached-windows](https://github.com/jefyt/memcached-windows)
- **Awesome Redis:**
 - <https://github.com/JamzyWang/awesome-redis>
 - <https://github.com/weyoss/awesome-redis>
 - <https://github.com/rmacduff/awesome-redis>
- **Türkçe Kaynaklar:**
 - <https://aws.amazon.com/tr/memcached/>
 - <https://aws.amazon.com/tr/redis/>